SYSTEMS REQUIREMENT DOCUMENT

DPP Data Pipeline

GROUP **RNN**

PASCUAL, NICANOR FROILAN
RAMOS, NATHAN RAFAEL
TORRES, ALOYSIUS REVE

To be served to:
PHILIPPINE MEDIA MONITORING LABORATORY (PMML)

PROJECT BLOG:
https://cs191-2627a-rnn.github.io

CS 191 WFR-1
Prof. LIGAYA LEAH FIGUEROA

Web App Requirements Model

I. Content Model

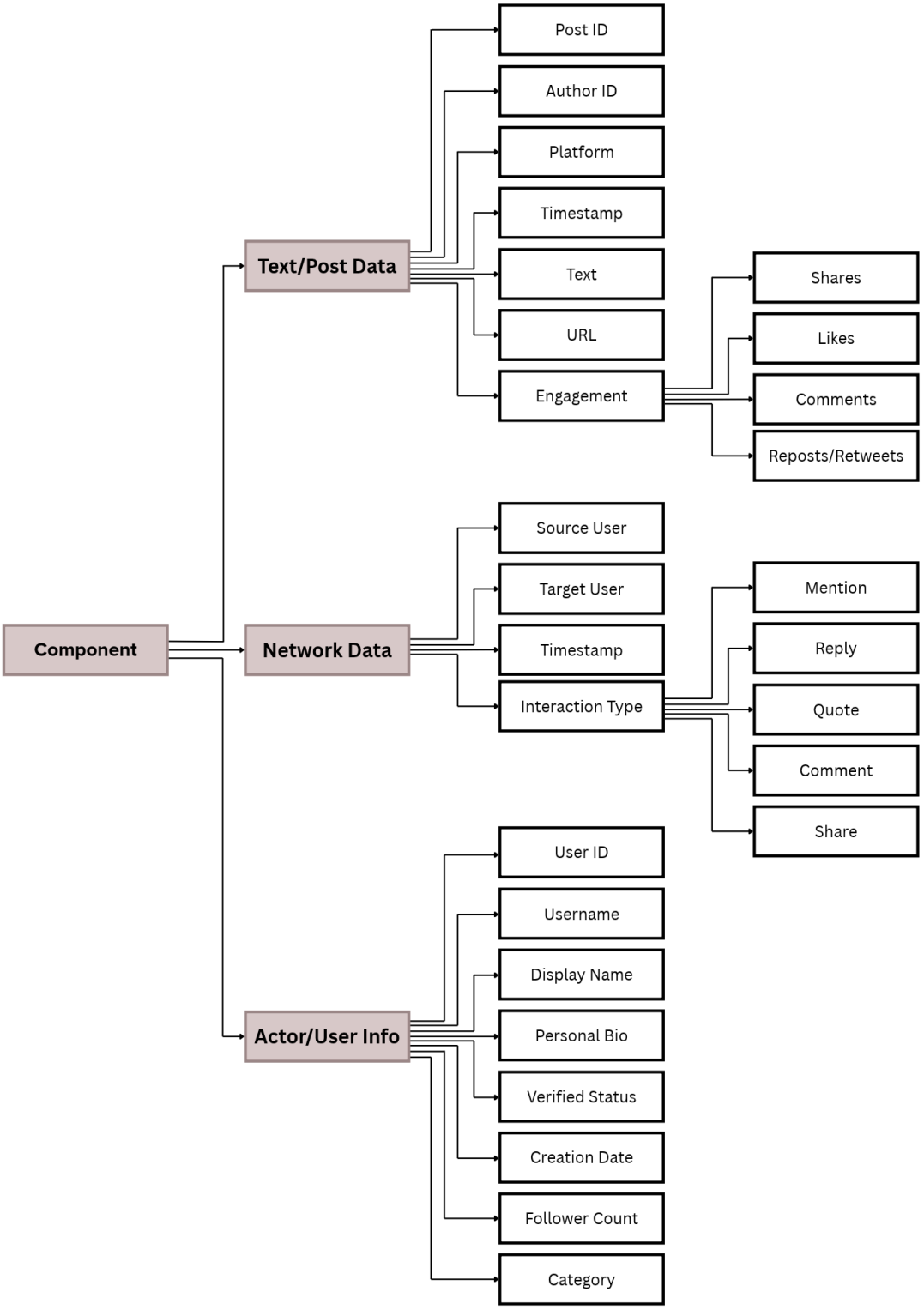


Figure 1: Data Tree of Content Objects for the DPP Data Pipeline (Grey - Content Objects, White - Data Items)

II. Interaction Model

The DPP Data Pipeline has three (3) actors:

- **Researchers** (Initiating): The product user. They require the pipeline to streamline the processing and analysis of data the DPP project needs.
- **System Administrators** (Initiating): These are users who need to have the ability to manage (turn on, shut down, monitor, rollback, etc.) the pipeline manually if needed.
- **Platform APIs** (Participating): Actors from which data will be collected from.

More specifically, the final product has the following use cases based on what each actor requires / contributes:

Researchers	
Use Case	Description
Input Data Into Pipeline	The pipeline can take in data from platform APIs the researchers pass onto it.
Parse and Process Datasets	Automate the raw text cleaning and processing through the pipeline.
Automate Data Validation	The pipeline should be able to check if there are missing key fields in the exported data.
Source and Temporal Analysis	Analyze the trends and engagement of posts through different time windows.
Network Analysis	Analyze through the use of network graphs the connections between data.
Message Analysis	Analyze the data to determine which keywords/phrases are most visible.
Visualize Analysis Through Graphs	Project the analyzed data through plotted graphs, mapping relevant relationships and networks.
View Analysis Through Dashboard	The user should be able to see the data through an interactive dashboard interface for exploration.
System Administrators	
Use Case	Description
Manual Configuration	The administrator can make changes to the system settings depending on need.
System Monitoring	A view where the administrator can watch over the system and any errors that may occur.

Platform APIs	
Use Case	Description
Retrieve Social Media Data	The system requests data from Apify or through the platform APIs themselves.
Return Extracted Data	The service returns the requested data which can be utilized for pre-processing by the system.

Table 1:Use Cases from each Actor

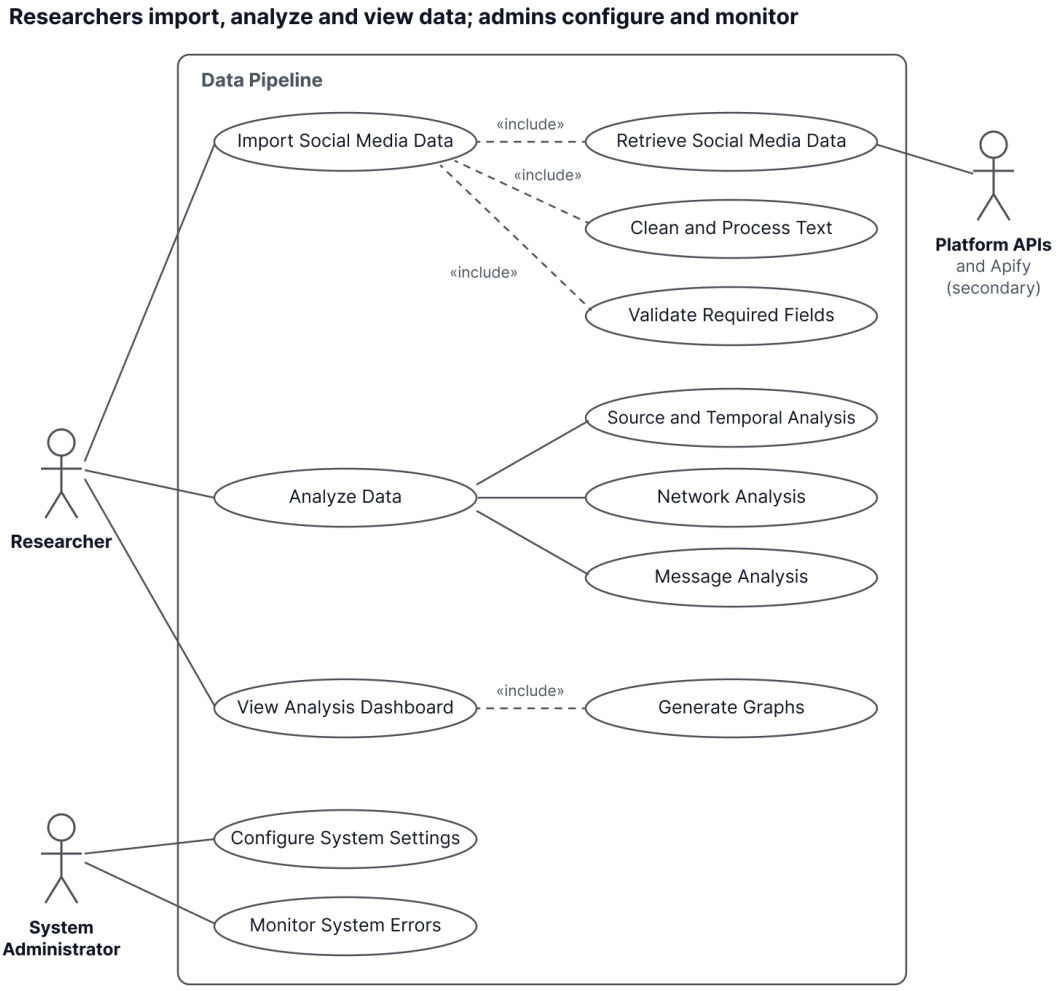


Figure 2: Use-case diagram for the DPP Data Pipeline

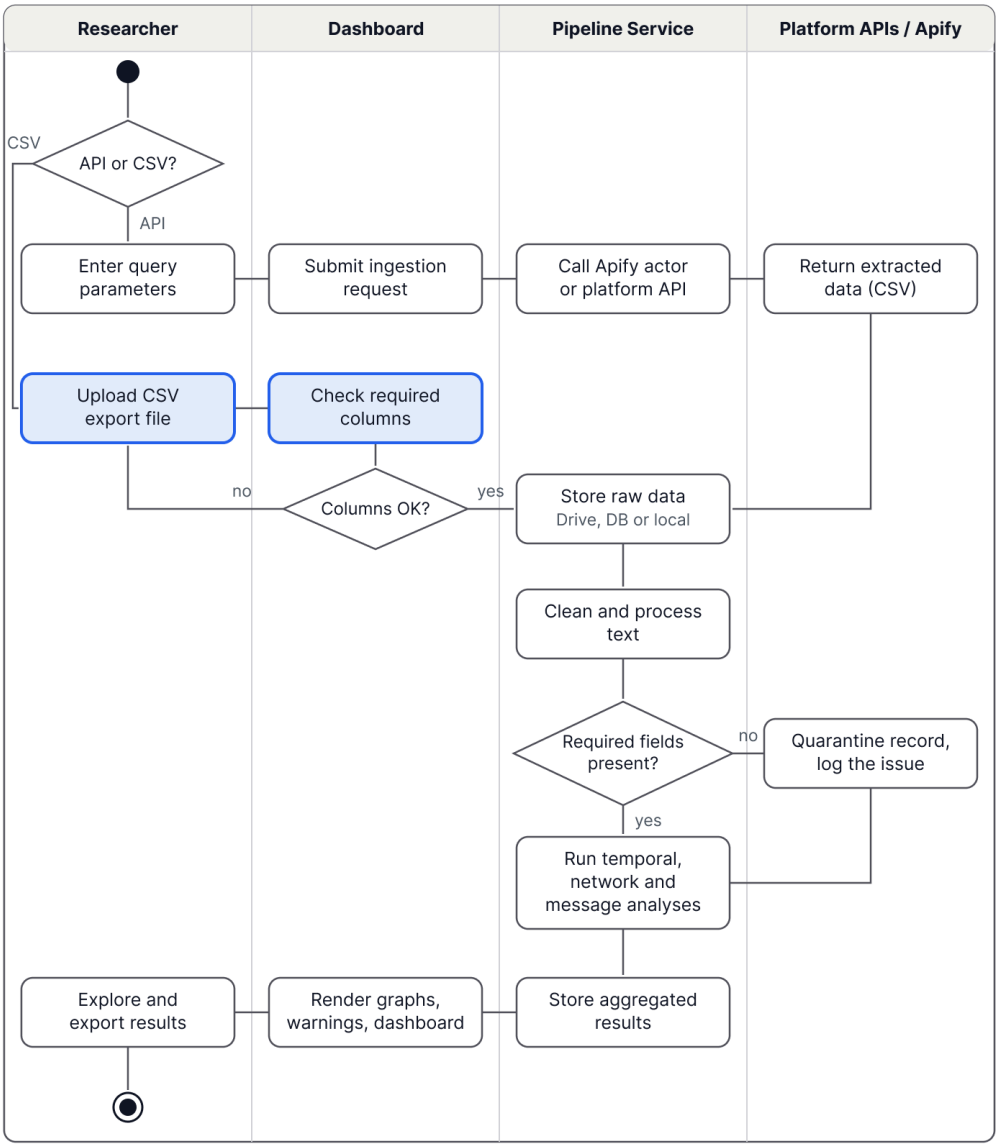


Figure 3: Swimlane Diagram

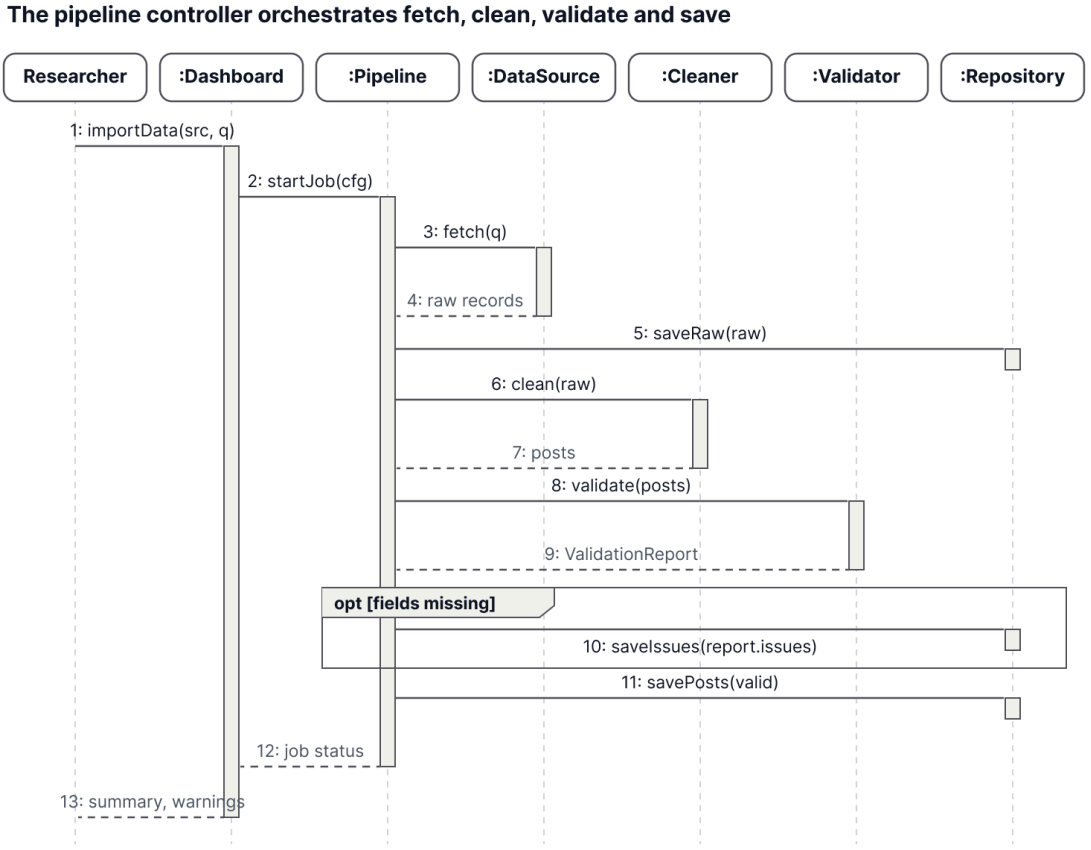


Figure 4: Sequence Diagram

III. Configuration Model

Subsystems and Allocations		
Computing Environment	Modules / Components	Responsibilities
User Machine	User Interface	Upload datasets, configure analysis parameters, and view interactive visualizations.
Web App Server	App Backend, App Frontend, API	Handle user requests, validate uploads, coordinate processing workflows, and return results to the client.
Processing Server	Data Processor, Data Analyzer, Visualization Generator	Parse, clean, and normalize datasets, perform analysis, and construct visuals.
Database / File Storage	Data Storage, Results Storage	Store uploaded datasets, cleaned records, and analyzed outputs.
External Services	Apify Actors, Supported Platform APIs	Supply extracted social media data when required.

Table 2: Possible Subsystems and their associated Modules and Responsibilities

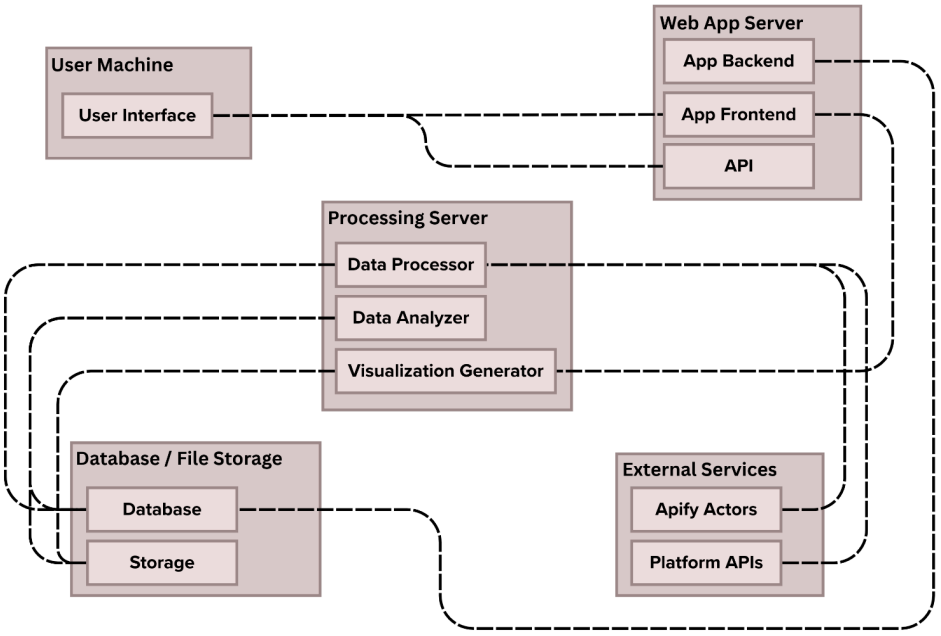


Figure 5: UML Deployment Diagram

Web App Architecture and Design

I. Architecture Design

The complete proposed system will adopt a client-server architecture with a three-tier logical structure and modular processing. These styles are selected in support of the primary purpose of streamlining the current data processing system of the Philippine Media Laboratory's Digital Public Pulse (DPP) project.

Client-Server Architecture

- The final system will follow a client-server architecture where users can interact with the pipeline through a web-based user interface. Going by the UML Deployment Diagram from the Configuration Model, the User Machine hosts the client-side interface, while the Web App Server provides the frontend, backend, and API modules that handle user requests.
- The frontend allows normal users to add datasets and explore interactive visualizations. System administrators, as users themselves, can additionally manage the system settings and monitor the pipeline for any errors that may occur.
- This architecture makes it so logic is separated from what the user can access, and facilitates the core requirements the client might need like consistent data ingestion, access control, and system maintenance.

Three-Tier Architecture

- The final system adopts a three-tier logical structure consisting of the following tiers. While they are shown as separate for the sake of separation of concerns, in practice, the Web App Server and Processing Server may be one continuous server:

  Presentation Tier. This serves as the user interface through which users interact with the pipeline. It includes the following interfaces: dataset upload, analysis visualization, and system logs.

  Processing Tier. This contains the modules responsible for the "data processing" portion of the system. The bulk of the work occurs with the Processing Server which hosts the processor, analyzer, and visuals generator.

  Data Tier. This consists of the Database / File Storage subsystem, which maintains records of the uploaded datasets and analysis results. The Processing Server writes processed data and generated results directly to this subsystem. The backend, in turn, retrieves stored information when responding to user requests from the frontend.

Modular Processing

- This means the datasets will pass through distinct stages of transformation and analysis. Each stage has a defined responsibility and produces results usable by succeeding stages:
    1. **Data Ingestion.** The system accepts datasets from the users through Apify and other supported platform APIs.
    2. **Data Pre-processing.** The data is cleaned, normalized, and standardized to establish consistent formats for further analysis. This stage also checks dataset structures and their required fields to identify invalid formats.
    3. **Data Analysis.** The system performs the three types of analysis as required by the client.
    4. **Visualization Generation.** Analytical outputs are given visual representations through the use of algorithms. These visuals include graphs, charts, and network visualizations.
    5. **Persistence and Presentation.** Outputs and datasets are stored for later retrieval, with the analysis outputs presented through the interactive dashboard.

- This modular structure allows each subunit to be independent from each other, allowing simpler maintenance and testing.

Quality Attribute Considerations

The architecture is designed to address the technical quality attributes identified by Pressman:

- **Usability.** Separating the presentation tier from processing responsibilities allows the interface to be developed and refined without requiring changes to the processing and analysis algorithms. This also ensures the user cannot accidentally interact with the "inner workings" as this could lead to further complexity, which affects usability.

- **Functionality.** The architecture supports the required functions through specialized modules. These include the earlier mentioned modules in the *Modular Processing* section.

- **Reliability.** This is addressed through consistent data and error handling by the system. Within the system itself, dataset validation deals with the invalid datasets, while error handling is resolved through the system administrator's monitoring.

- **Efficiency.** The architecture separates the computationally-intensive processing from the user-facing interface. This reduces the need for the backend to perform intensive analytical operations while also handling interactive requests.

- **Maintainability.** The separation of concerns as mentioned in *Modular Processing* helps with the continued maintainability of the system. If one part of the pipeline were to fail, then the other parts should not be affected. This also enables separate testing of these parts of the pipeline, which helps with maintainability and reliability.

Other Architectural Decisions

- Included in this section are the architectural decisions that were not explicitly described in the earlier sections.
    1. **Establishment of a Common Schema Entity.** This is inline with the content objects (Post data, Network data, and Actor / User information) as specified in the Data Tree (Figure 1). This decision allows integration between different platform APIs and enables consistent operation of analytical modules.
    2. **Persistence by the Processing Server.** The Processing Server will write directly to the Database / File Storage, before the Web App Server's backend will retrieve results for presentation. While slower, as it retrieves data from some storage space, this avoids sending large amounts of data through the backend solely for storage.
    3. **Administrative Configuration and Monitoring.** The system will allow a separate Administrative interface dealing with system configuration and monitoring. This allows authorized users to configure and watch over the system instead of allowing every user, increasing security. However, the tradeoff is that any critical error a normal user has will have to go through the administrator to resolve.

II. Content (Data) Design

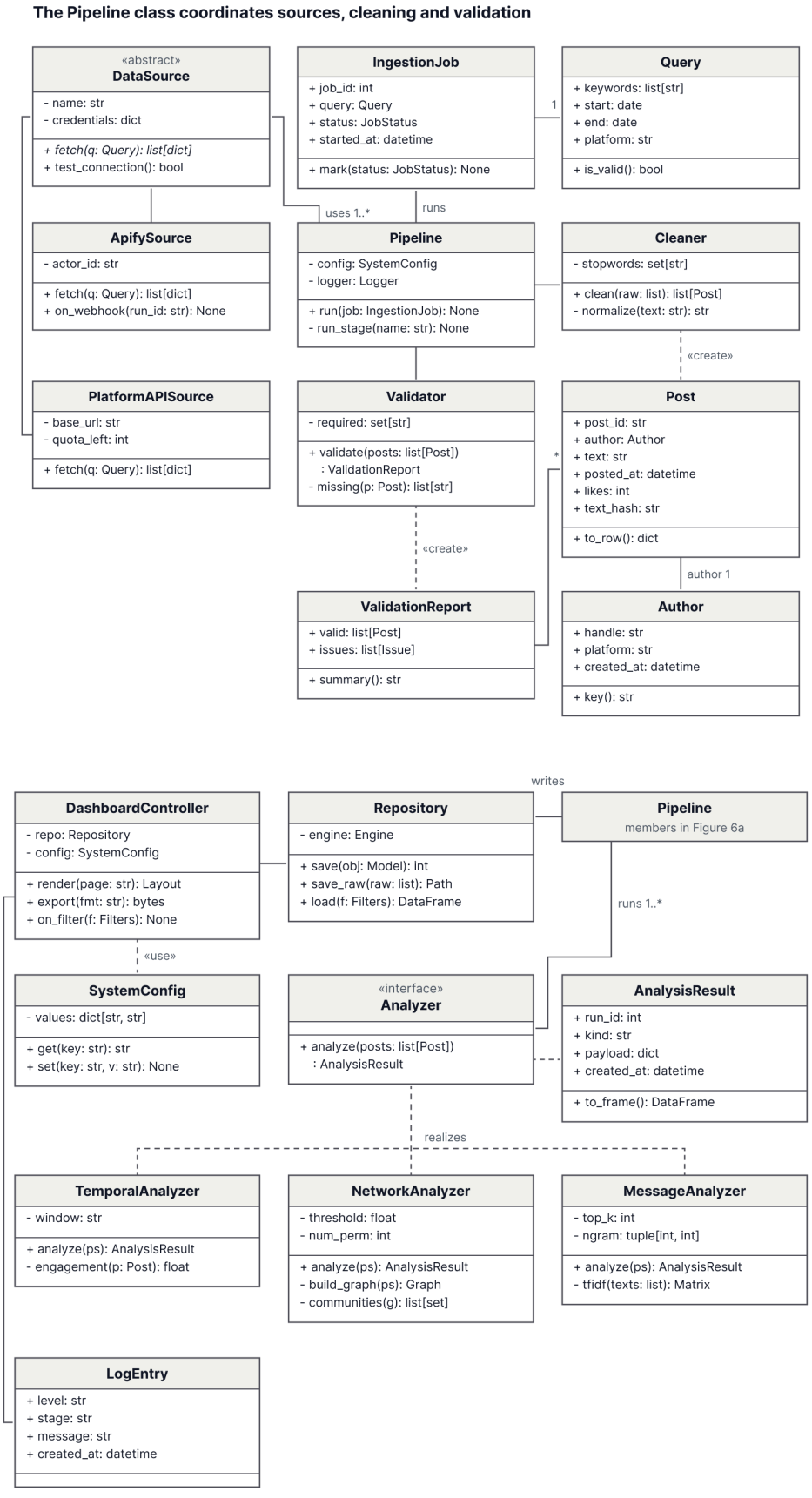


Figure 6: Class Diagrams

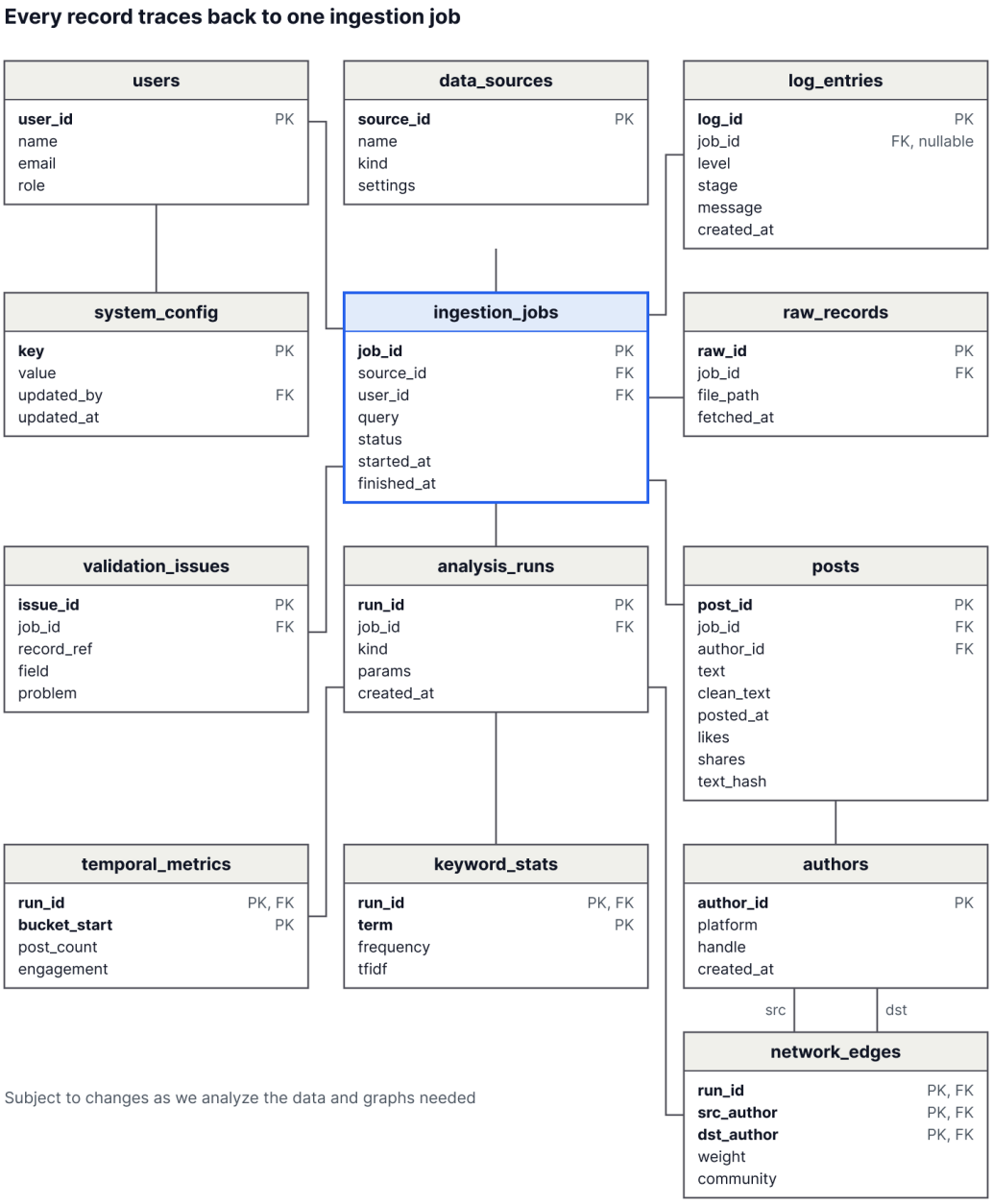
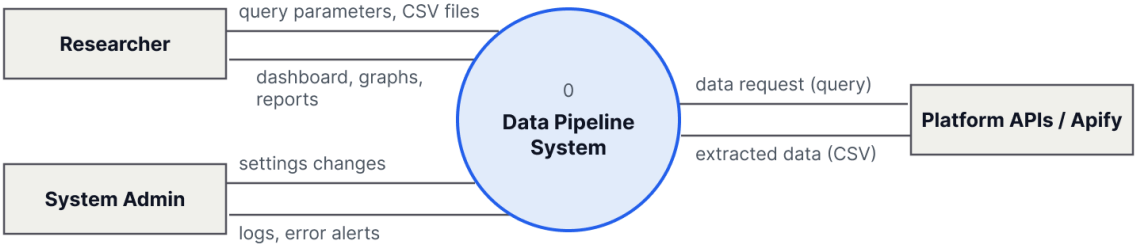
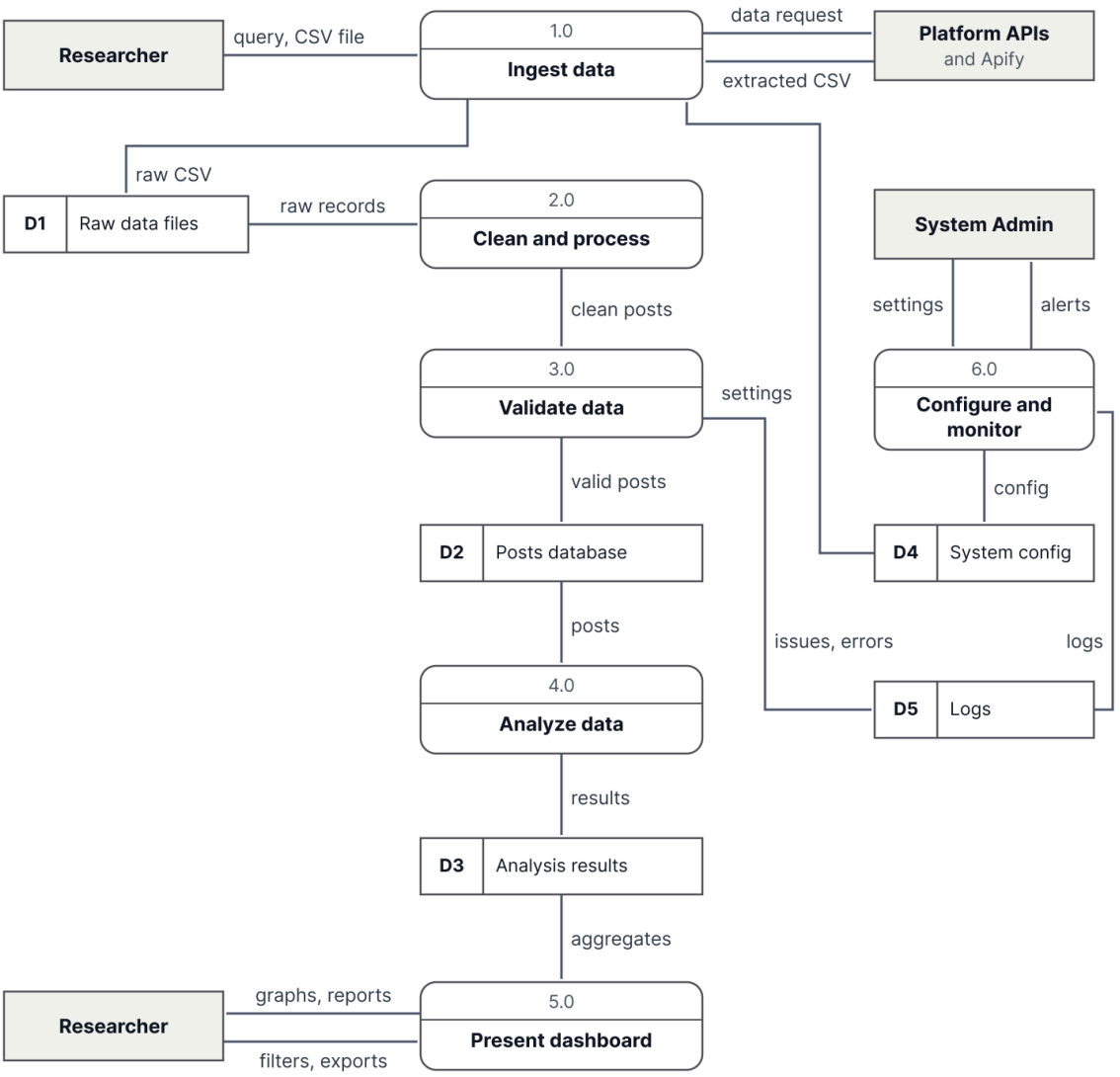


Figure 7: Entity-Relationship Diagram

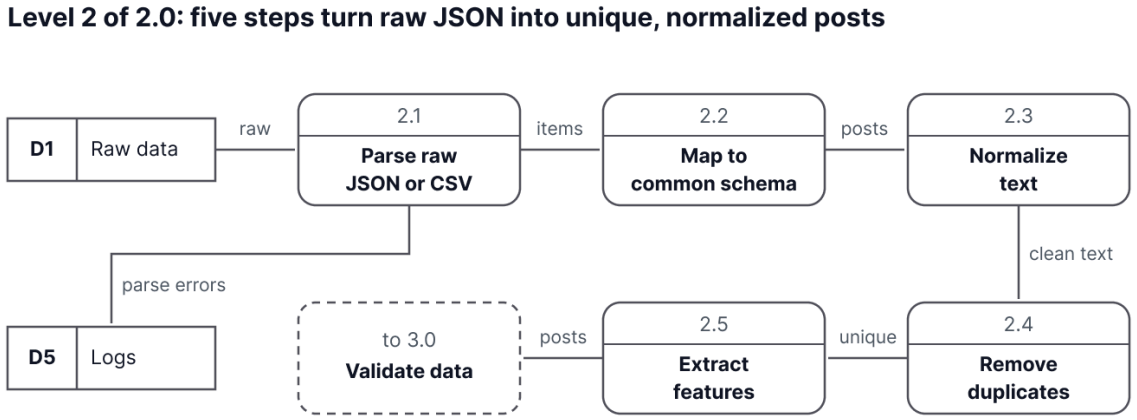
Level 0: three external entities exchange data with one system



Level 1: data moves through six processes and five stores



Processes 1.0 to 5.0 also write log events to D5; those flows are omitted for clarity.



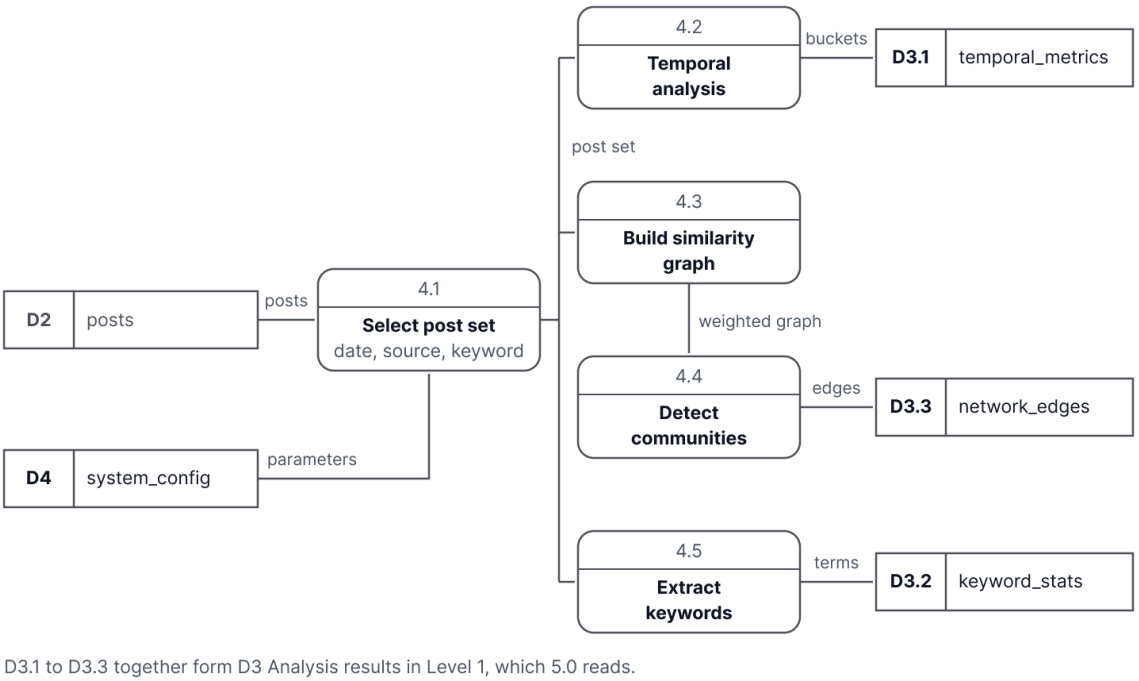


Figure 8: Level 0 to 2 Data Flow Diagrams

III. Component-level Design

Specify the system components – define the data structures of all local data objects and algorithmic/procedural detail of all processing within a component (use the sub-questions below as a guide). Include interfaces (see Figure 8.5) and dependencies (e.g., sharing, flow, constrained - see Section 9.5.2). Will you be using any COTS components? (See chapter 10). Refer to Chapter 8 and Chapter 10 as you discuss design concepts that you have implemented.

- **Algorithms**

Describe complex functional/processing details of the web app through a UML activity diagram.
For example, when computing a motion trajectory for an animate figure in a game, you may use some numerical or computer-graphics algorithms. Will the path coordinates be pre-computed and stored in a look-up table or will they be computed using a spline interpolation algorithm? Or, when assessing stock market movements, you may be using statistical algorithms.

- **Data Structures**
  Does your system use any complex data structures, such as arrays, linked lists, hash tables, or trees? Specify your criteria for deciding what data structure to use, e.g., performance vs. flexibility.

IV. Interface Elements / Usability Design

The following set of graphics (Figure A to G) below will be the template for how the interface may look like for the project. Each page has the navigation links at the top since the content should be the priority. To indicate which interface the user is in, the corresponding navigation page title will appear on the header, and will disappear in the navigation bar.

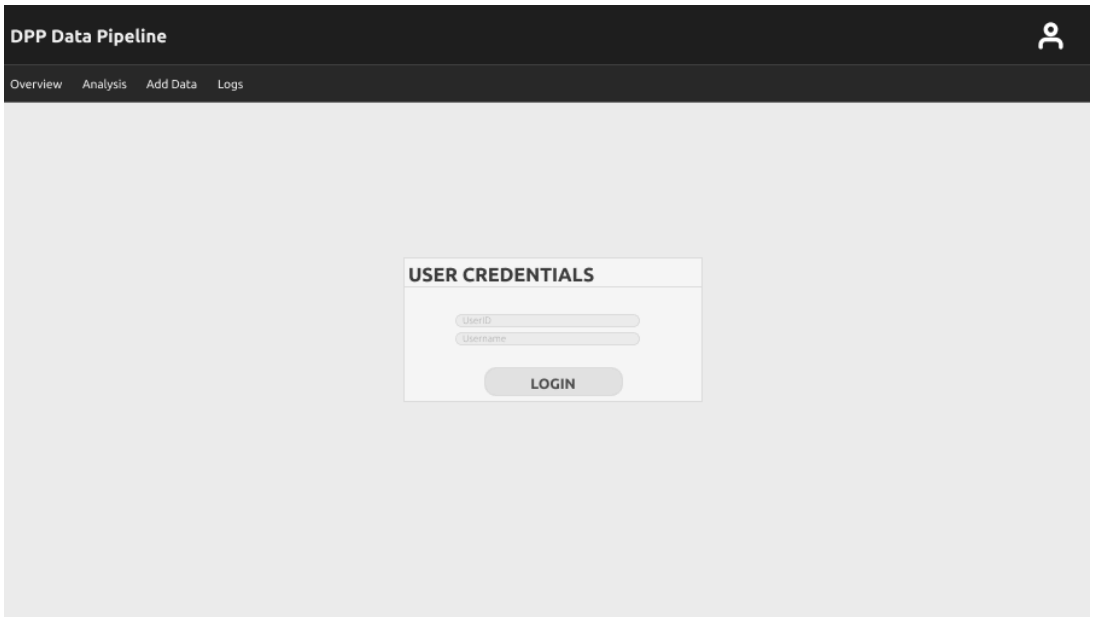


Figure A: Login Interface

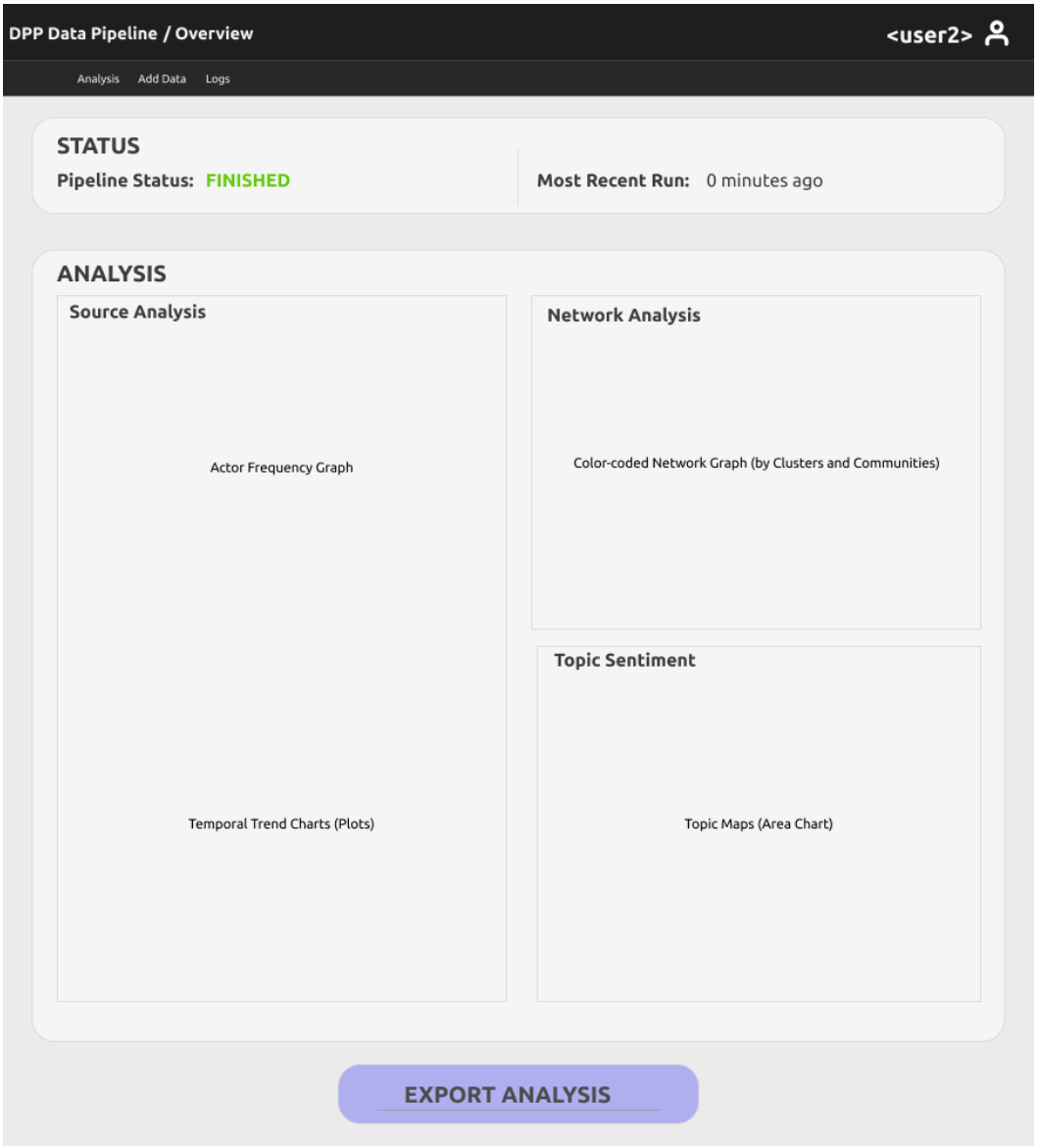


Figure B: Overview Interface



Figure C: More Detailed Analysis Interface

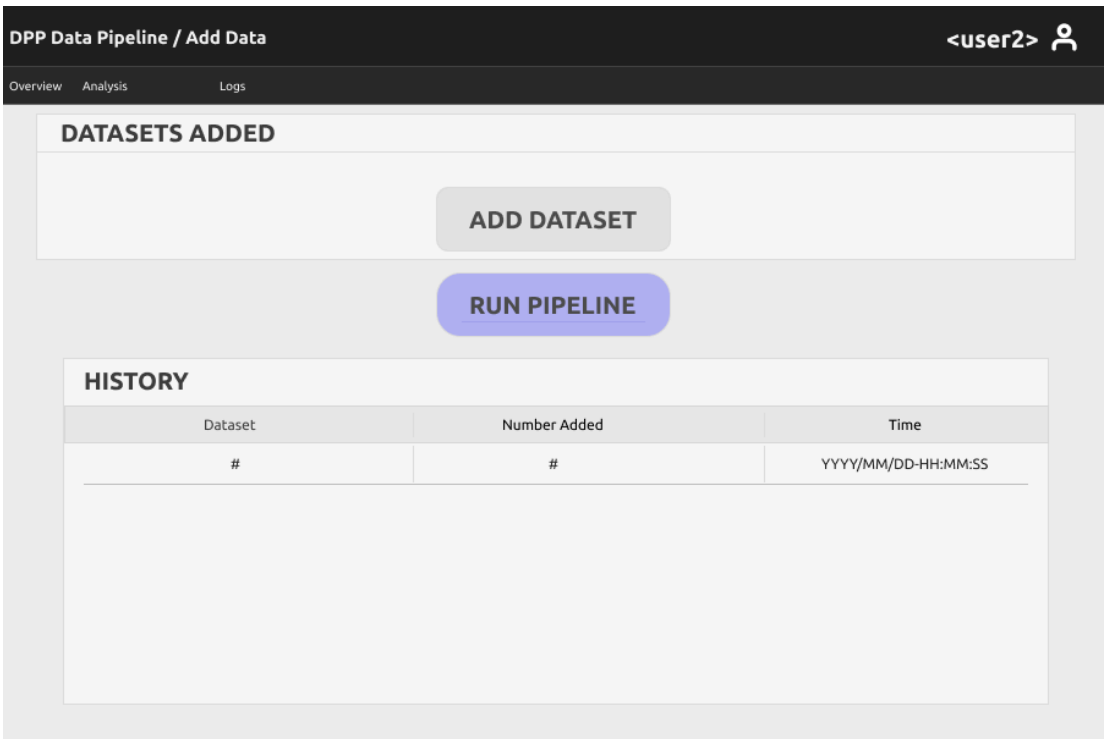


Figure D: Add Dataset Interface

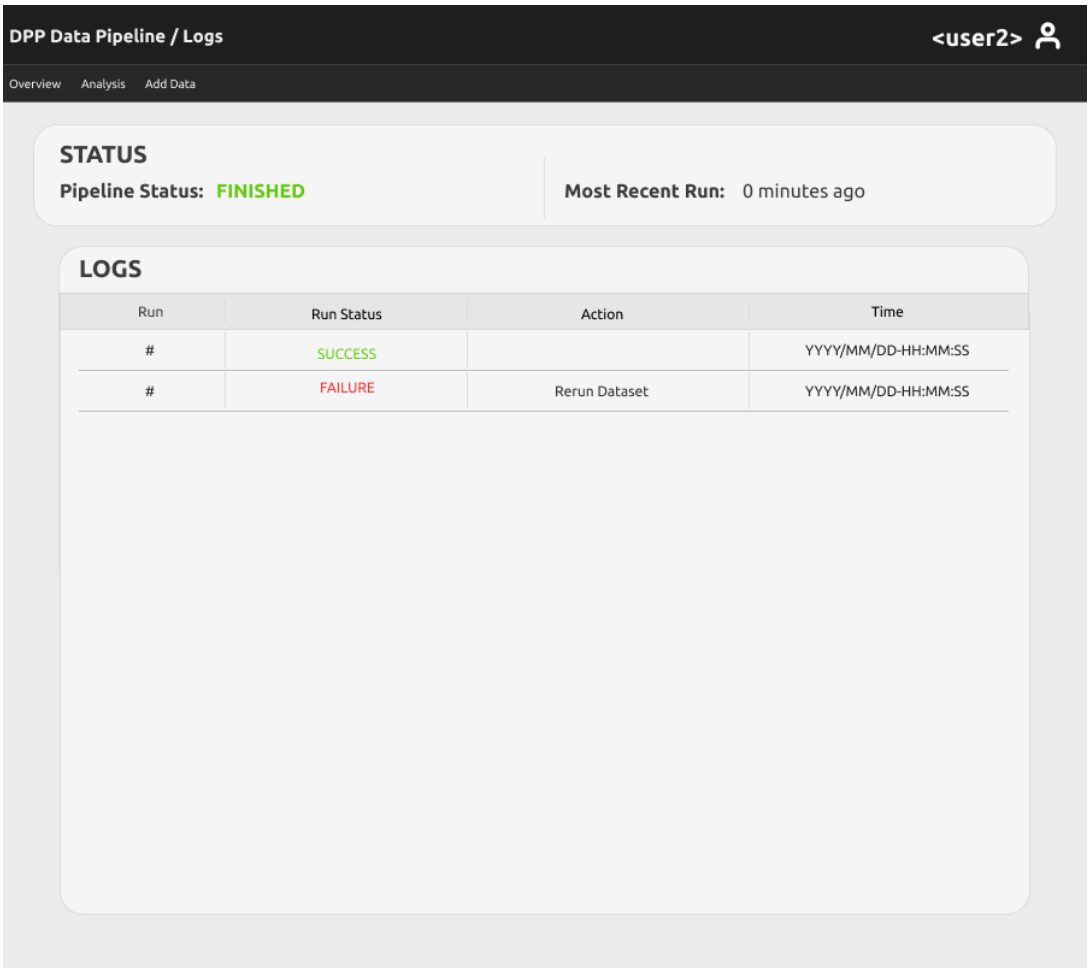


Figure E: Pipeline Logs Interface

In addition, the system administrator, who is a privileged user, has access to 2 more navigation pages dealing with system control. The administrator page can be distinguished by its dark blue header contrasting the black header for normal users:

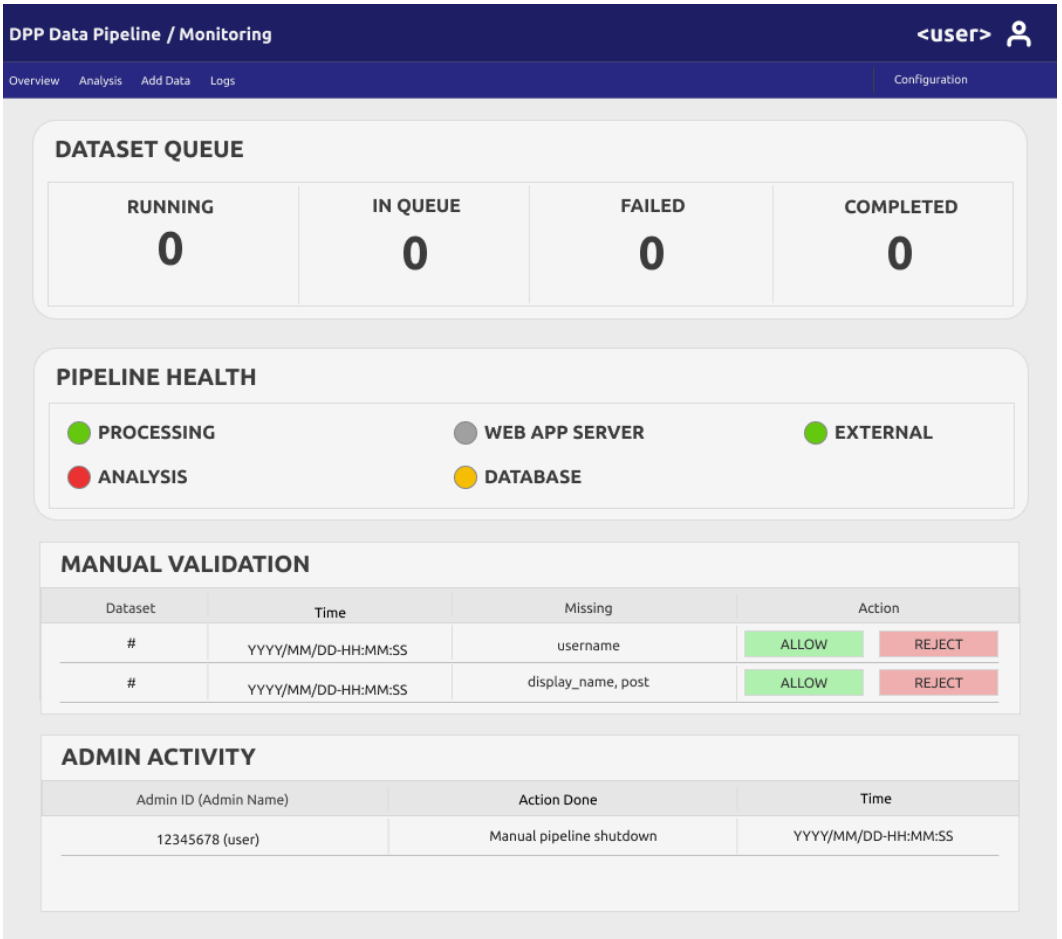


Figure F: System Monitoring Interface

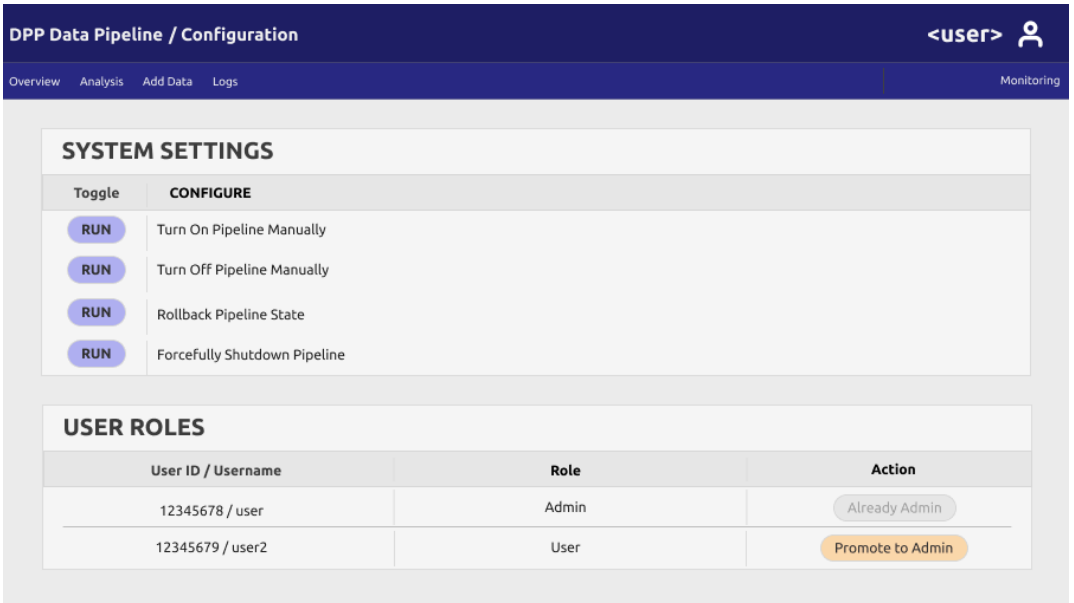


Figure G: System Configuration Interface

Going by the issues proposed in Section 11.4.3 of Pressman's *Software Engineering: A Practitioner's Approach*, the design addresses these issues through the following:

- **Response Time**: The interface is expected to respond relatively quickly when navigating between individual interfaces, owing to their decoupled design. However, the software's processing time is expected to be comparatively longer due to the nature of the data processing pipeline, which can accept and process large datasets sequentially. This behavior is considered acceptable within the project's intended context. Nevertheless, we aim to maintain consistent and predictable processing times for comparable dataset sizes and workloads.

- **Help Facilities**: This will be expanded upon in the future as we get more user feedback to determine what makes the interface hard to use. In practice, the design can be expanded upon to clearly indicate what each color means, what each state of the pipeline is, and what each section does. This can be done either through a dedicated interface serving as a user manual or including at the top (or bottom) of each section what they are responsible for in each interface.

- **Error Handling**: Errors occurring within the interface or processing pipeline will be recorded in the system administrator's Monitoring interface. This allows administrators to investigate errors and take appropriate corrective actions depending on their nature. Possible responses include restarting the system, reverting to a previous state when recovery mechanisms permit, or manually validating a dataset when an error results from a missing field. Regular users can identify pipeline failures through the status indicators displayed in the *Overview* or *Logs* interfaces, where an *Error* status indicates that an issue has occurred. Users can then contact the system administrator for assistance in diagnosing the problem.

- **Menu and Command Labeling**: Menu items and commands are labeled to clearly communicate their intended functions. For example, the *Run Pipeline* command in the *Add Datasets* interface initiates pipeline execution for the submitted datasets, provided that no other pipeline execution is currently queued. In addition, color coding is used to communicate the purpose of certain actions. For example, the system administrator's Monitoring interface uses green *Allow* and red *Reject* buttons for manually validating datasets.

- **Application Accessibility**: The interface is designed to accommodate the context in which the client will use the application. Several design considerations have been incorporated to improve accessibility, including larger text sizes, color-coded status indicators and buttons, and a sectioned layout that allows users to locate and view the information they need more easily. Color coding is supplemented by textual labels, such as *Error*, *Allow*, and *Reject*, so that the meaning of important statuses and actions is not conveyed through color alone.

- **Internationalization**: English is the primary language of the interface, allowing many users to understand the information presented through the website in addition to interpreting its diagrams and charts. However, since the project is primarily intended to serve the current client's needs, internationalization is not considered a concern as of current.

Regarding the **Aesthetic Design** of the web app, the only issue, going by Pressman's guidelines, is with respect to the scrolling. The design prioritizes navigation over minimal scrolling, since this is more appropriate for organizing and accessing the interface's content. However, all the other guidelines were followed:

- There is a reasonable amount of white space in between the page sections.
- Rather than placing the navigation bar on the side, as stated earlier, it was placed at the top so that the content was the emphasis.
- The content flows from top-left to bottom-right.
- There is a clear geography in the pages. Navigation links are grouped together in the navigation bar and similar content sections are grouped together in their own sections.

With regards to the visual design itself, readability was favored over decorative elements. This meant ensuring there is sufficient contrast between the text from the background to improve legibility. Furthermore, the header color itself is part of the design, as aside from communicating to the current user their privilege level, it adds a bit of color to the interface.

To explain the **Navigation Design,** we can look at how "friendly" the design is with navigation, i.e., if the user requires less effort to move compared to doing anything else, then we can say it's quite user-friendly for navigation. Included below is a table of some typical usage scenarios and how many clicks are required to accomplish their specified task:

Possible Usage Scenarios		
Navigation Presses	Clerical Data Entry	Usage Scenario
2	2	A user inputs data into the pipeline and waits for the analysis to complete.
0	0	A user logs in to view the *Overview*.
3	2	A user inputs data into the pipeline, sees an error, and looks at the logs.

2	4	A user enqueues another dataset immediately after enqueueing the first one, and waits for the analysis to complete.
1	2	A user goes into the *Analysis* section, and uses the filter mechanism of *Network Analysis*.
1	1	The administrator does manual data validation for a single invalid dataset.
1	1	The administrator goes into the system configuration and turns off the pipeline manually.
1	1	The system administrator goes into the system configuration and elevates a user to administrator.

Table 3: Usage Scenarios a user or administrator might do.

In these possible usage scenarios, we have 11 navigation presses and 13 clerical data entries (button click, add dataset, etc.) done. This means below 50% (45.8333%, to be exact) goes to navigation, with the rest going into data entries.

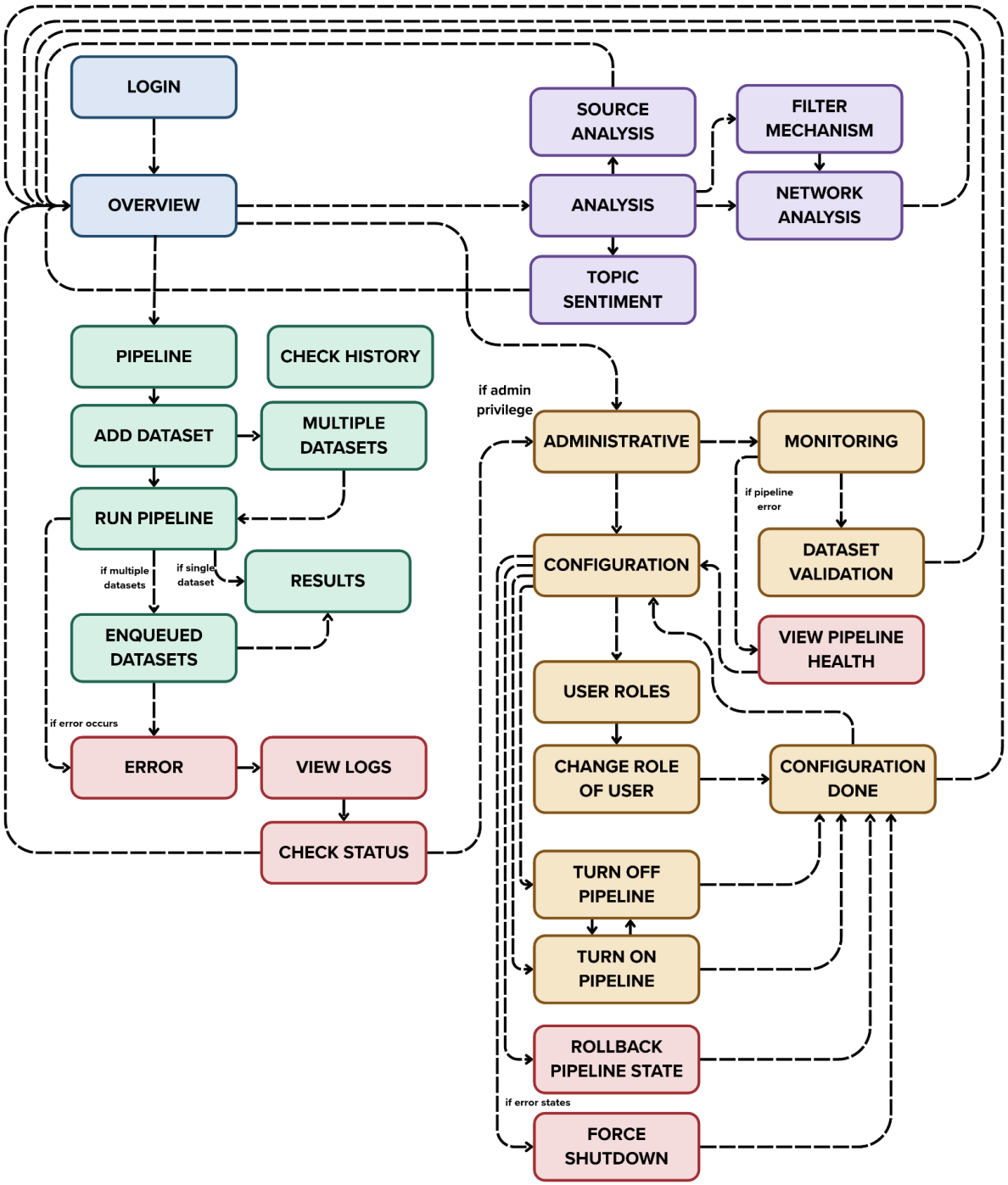


Figure H: Combined Navigation Semantic Unit

REFERENCES

Bunquin, J.B., Gaw, F., Baldo-Cubelo, J.T., Paragas, F., & San Pascual, M.R. (2022). *Digital Public Pulse: 2022 Philippine General Elections.* Philippine Media Monitoring Laboratory, College of Mass Communication, University of the Philippines.

Pressman, R. S. (2010). *Software Engineering A Practitioner's Approach*. McGraw-Hill Higher Education.